量化交易进阶:第50天 – 深度学习因子模型(Deep Factor Model)

原创 逻辑与流 逻辑与流 2025年11月15日 00:16 北京 标题已修改 听全文

本文系统讲解如何使用MLP(多层感知机)和LSTM(长短期记忆网络)构建深度学习因子模型,预测未来收益率。从传统线性模型的局限性出发,阐述深度学习模型如何通过多层非线性变换和时序记忆机制自动学习复杂因子关系。通过完整的Python代码实战,演示如何使用Tushare获取贵州茅台历史数据,构建18个多维度因子,训练优化的LSTM模型预测未来5个交易日的涨跌趋势。探讨为什么深度模型能捕捉非线性因子关系、LSTM时间窗口长度如何选择、多股票场景下如何引入行业和规模embedding等核心问题,为读者构建从传统量化到智能量化的技术升级路径。

学习时间

- **理论学习**: 20分钟
- **代码实践**: 20分钟
- **案例分析**: 20分钟
- **总计**: 60分钟

一、背景介绍

传统量化因子（如 PE、PB、ROE、动量、波动率等）大多依赖线性模型：

$$r_{t+1} = \beta_0 + \sum_i \beta_i x_{i,t} + \epsilon_t$$

但市场关系往往是非线性、时变的，因此我们可以使用深度学习模型（MLP、LSTM）来学习这种复杂关系，从而更好地预测未来收益率。

二、MLP 与 LSTM 是什么?

（1）MLP（Multilayer Perceptron，多层感知机）

MLP 是最经典的前馈神经网络（Feedforward Neural Network）。

结构是这样：

```
输入层（特征 x1,x2,...,xn）
      ↓
隐藏层1 -- 非线性函数 ReLU/Tanh
      ↓
隐藏层2 -- 非线性函数 ReLU/Tanh
      ↓
输出层（预测收益 / 概率）
```

核心公式：

$$h_1 = \sigma(W_1 x + b_1)$$

$$h_2 = \sigma(W_2 h_1 + b_2)$$

$$\hat{y} = W_3 h_2 + b_3$$

每一层都用非线性函数（如 ReLU、tanh），使得模型能拟合任意复杂的函数关系。

所以，MLP 本质上就是：

"一堆线性变换 + 非线性函数"的堆叠，可以学习非常复杂的特征组合关系。

（2）LSTM（Long Short-Term Memory，长短期记忆网络）

LSTM 是一种专门用于时间序列建模的神经网络。

它的核心思想是：

当前的预测，不仅依赖当前特征，还依赖过去一段时间的特征变化模式。

LSTM 内部结构：

- 输入门（决定读入多少新信息）
- 遗忘门（决定遗忘多少旧信息）
- 输出门（决定输出多少记忆）

数学上，它通过门控机制控制"记忆流"，从而解决传统 RNN 无法处理长期依赖的问题。

适合场景：

- 股票、期货、汇率的历史价格走势；
- 因子在时间上的演变（如动量、波动、ROE 的变化趋势）。

三、为什么它们能学习"非线性""时变""复杂关系"?

1. 非线性关系

传统线性模型（如逻辑回归、线性回归）只能学到：

$$r = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots$$

比如：

- "PE 越低越好" 是线性的；
- 但"PE 太高或太低都不好" 是非线性关系。

MLP 通过 ReLU / Tanh 激活函数 可以表达出：

$$r = f(x_1, x_2, \ldots, x_n)$$

其中 f 是任意复杂的函数，能拟合 U 型、S 型甚至混合模式。

2. 时变关系（LSTM）

市场规律不是固定的，比如：

- 疫情时防疫股强；
- 科技周期时成长股强。

这意味着：

相同的因子，在不同时期的作用不同。

LSTM 通过"记忆单元"在时间维度上捕捉这种变化：

它能学习到"上一阶段的趋势 + 当前环境"如何共同影响未来收益。

简单比喻：

- MLP 看的是"静态快照"，
- LSTM 看的是"动态影片"。

3. 因子交互（Feature Interaction）

在金融数据中，特征间往往有强烈交互：

- ROE 高 + PB 低 = 价值股；
- 动量强 + 波动低 = 稳健上涨股。

传统模型无法自动挖掘这种乘积或组合关系，

而 MLP/LSTM 能自动学出这些高维交互项。

举个形象的例子

假设我们预测未来5日收益：

- 因子有 PE、PB、ROE、动量；
- 关系是复杂的，比如：
  - PE 太高 → 风险；
  - 但 ROE 高 时 PE 高也没关系；
  - 过去10天动量上升 → 持续上涨概率大。

传统线性模型只能做：

$$r = a_1 PE + a_2 PB + a_3 ROE + a_4 \text{ 动量}$$

MLP/LSTM 能学到：

$$r = f(PE, PB, ROE, \text{动量}, \text{历史动量趋势})$$

其中 f 可以是任意复杂函数。

这就是"非线性"和"时变"的强大之处。

总结对比

模型类型	特征	是否捕捉时序	是否建模非线性	优点
线性回归	静态	❌	❌	可解释性强
MLP	静态	❌	✅	能拟合复杂因子关系
LSTM	动态（时间序列）	✅	✅	能捕捉市场时变规律

四、思路流程

我们以 A 股为例，用 tushare 下载股票因子数据 + 收益数据：

1. 下载每日行情、基本面指标（PE、PB、ROE、动量等）；
2. 构造样本特征 X_t，目标为未来5日收益率 y_{t+5}；
3. 使用 LSTM 拟合关系；
4. 评估 AUC、IC、回测收益。

五、代码实战

基于 LSTM 的股票趋势预测程序，通过历史价格、财务数据和动量因子，预测未来5个交易日的涨跌趋势。已进行模型结构、训练策略和正则化优化。

核心信息

目标股票：贵州茅台（600519.SH）

神经网络模型：LSTM（长短期记忆网络）

- 单层 LSTM，隐藏单元：32（优化后降低，减少过拟合）
- 全连接层：32 → 16 → 1（添加中间层和 Dropout）
- Dropout 率：0.3（防止过拟合）
- 序列长度：10 天（使用过去 10 个交易日数据）

- 激活函数：ReLU + Sigmoid

使用的因子（共 18 个）：

1. **基础价格因子（4 个）**：
   - 开盘价（open）、最高价（high）、最低价（low）、收盘价（close）
2. **成交量因子（1 个）**：
   - 成交量（vol）
3. **财务因子（3 个）**：
   - 市盈率（PE）、市净率（PB）、净资产收益率（ROE）
4. **动量因子（10 个）**：
   - 收益率动量（5 个）：1 日、3 日、5 日、10 日、20 日收益率
   - 价格动量（3 个）：5 日、10 日、20 日价格动量
   - 成交量动量（2 个）：5 日、10 日成交量变化率

数据范围：2020 年 1 月 1 日至当前日期，约 5 年 10 个月的历史数据

预测目标：未来 5 个交易日的涨跌趋势，输出上涨概率与下跌概率

优化措施

1. **模型结构优化**：
   - 隐藏层维度从 64 降至 32
   - 添加 Dropout（0.3）防止过拟合
   - 全连接层增加中间层（32→16→1）
2. **训练策略优化**：
   - 早停机制（patience=10）
   - 学习率自动调度（ReduceLROnPlateau）
   - 梯度裁剪（max_norm=1.0）
   - L2 正则化（weight_decay=1e-4）
3. **类别不平衡处理**：
   - 自动计算类别权重
   - 使用带权重的损失函数（BCEWithLogitsLoss）
4. **训练监控**：
   - 实时监控训练和验证 AUC
   - 自动保存最佳模型

工作原理

1. 使用过去 10 个交易日的 18 个因子作为输入
2. 通过 LSTM 学习历史数据的时序模式
3. 动量因子捕捉不同周期的价格趋势和成交量变化
4. 输出未来 5 个交易日的上涨概率
5. 根据概率判断看涨或看跌

```
import tushare as ts
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_auc_score
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from datetime import datetime, timedelta

# ===== 初始化 tushare =====
ts.set_token('')
pro = ts.pro_api()

# ===== 1. 获取行情与财务数据 =====
# 获取最新数据（到今天的日期）
today = datetime.now().strftime('%Y%m%d')
start_date = '20200101'  # 从2020年开始
print(f"获取数据范围: {start_date} 至 {today}")

df = pro.daily(ts_code='600519.SH', start_date=start_date, end_date=today)
df = df.sort_values('trade_date')

# 获取每日市盈率/市净率等（在 daily_basic）
basic = pro.daily_basic(ts_code='600519.SH', start_date=start_date, end_date=today,
                        fields='ts_code,trade_date,pe,pb')

# 获取财务指标（ROE）
fin = pro.fina_indicator(ts_code='600519.SH', fields='end_date,roe')
fin.rename(columns={'end_date': 'trade_date'}, inplace=True)

# 合并所有数据
data = df.merge(basic, on=['ts_code', 'trade_date'], how='left')
data = data.merge(fin, on='trade_date', how='left')
data = data.sort_values('trade_date')
data = data.ffill().fillna(0)  # Forward fill, then fill remaining NaN with 0

# ===== 2. 构造特征与目标 =====
```

```python
# 构造动量因子
data['momentum_1d'] = data['close'].pct_change(1)  # 1日收益率
data['momentum_3d'] = data['close'].pct_change(3)  # 3日收益率
data['momentum_5d'] = data['close'].pct_change(5)  # 5日收益率
data['momentum_10d'] = data['close'].pct_change(10)  # 10日收益率
data['momentum_20d'] = data['close'].pct_change(20)  # 20日收益率

# 价格动量（当前价格相对于N天前的比率）
data['price_momentum_5d'] = data['close'] / data['close'].shift(5) - 1  # 5日价格动量
data['price_momentum_10d'] = data['close'] / data['close'].shift(10) - 1  # 10日价格动量
data['price_momentum_20d'] = data['close'] / data['close'].shift(20) - 1  # 20日价格动量

# 成交量动量
data['vol_momentum_5d'] = data['vol'].pct_change(5)  # 5日成交量变化率
data['vol_momentum_10d'] = data['vol'].pct_change(10)  # 10日成交量变化率

# 构造目标
data['return_5d'] = data['close'].shift(-5) / data['close'] - 1
data['label'] = (data['return_5d'] > 0).astype(int)

# 删除包含NaN的行
# 注意：动量因子会在前20行产生NaN，目标会在最后5行产生NaN
data = data.dropna()

# 特征列表（基础特征 + 动量因子）
features = [
    # 基础价格和成交量
    'open', 'high', 'low', 'close', 'vol',
    # 财务指标
    'pe', 'pb', 'roe',
    # 动量因子
    'momentum_1d', 'momentum_3d', 'momentum_5d', 'momentum_10d', 'momentum_20d',
    'price_momentum_5d', 'price_momentum_10d', 'price_momentum_20d',
    'vol_momentum_5d', 'vol_momentum_10d'
]

X = data[features].values
y = data['label'].values

print(f"\n特征数量: {len(features)} (包含 {len(features)-8} 个动量因子)")
print(f"特征列表: {features}")

# 验证数据中没有NaN或inf
assert not np.isnan(X).any(), "X contains NaN values"
assert not np.isinf(X).any(), "X contains inf values"

# ===== 3. 数据标准化 =====
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```python
# ===== 4. 构建序列数据 =====
seq_len = 10
X_seq, y_seq = [], []
for i in range(len(X_scaled) - seq_len - 5):
    X_seq.append(X_scaled[i:i+seq_len])
    y_seq.append(y[i+seq_len+5-1])

X_seq = np.array(X_seq)
y_seq = np.array(y_seq)

print(f"总序列样本数: {len(X_seq)}")

# 保留最后10%用于验证，其余用于训练（用所有数据训练）
val_split_idx = int(len(X_seq) * 0.9)
X_train, X_val = X_seq[:val_split_idx], X_seq[val_split_idx:]
y_train, y_val = y_seq[:val_split_idx], y_seq[val_split_idx:]

print(f"训练集样本数: {len(X_train)}, 验证集样本数: {len(X_val)}")

# 保存最新的10天数据用于预测（原始数据，未标准化）
latest_10_days = X_scaled[-seq_len:]  # 最新的10天标准化数据
latest_dates = data['trade_date'].values[-seq_len:]  # 对应的日期

# ===== 5. 定义优化的 LSTM 模型 =====
class LSTMModel(nn.Module):
    def __init__(self, input_dim, hidden_dim=32, dropout=0.3):
        super().__init__()
        # 简化模型：降低隐藏层维度，增加Dropout防止过拟合
        # 注意：单层LSTM不支持dropout，dropout只在全连接层使用
        self.lstm = nn.LSTM(
            input_dim,
            hidden_dim,
            batch_first=True,
            num_layers=1
        )
        self.fc = nn.Sequential(
            nn.Linear(hidden_dim, 16),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(16, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        lstm_out, (h, _) = self.lstm(x)
        # 使用最后一个时间步的隐藏状态
        return self.fc(h[-1])

device = 'cuda' if torch.cuda.is_available() else 'cpu'
model = LSTMModel(input_dim=len(features), hidden_dim=32, dropout=0.3).to(device)

# 检查类别不平衡
pos_weight = (len(y_train) - np.sum(y_train)) / np.sum(y_train)
```

```
print(f"\n类别权重（正样本）: {pos_weight:.3f}")

# 使用带权重的损失函数处理不平衡
criterion = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([pos_weight], dtype=torch.float32).

# 降低学习率，增加L2正则化
optimizer = optim.Adam(model.parameters(), lr=5e-4, weight_decay=1e-4)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.5, patience

# ===== 6. 训练（带早停） =====
print("\n开始训练模型（使用所有历史数据）...")
X_train_t = torch.tensor(X_train, dtype=torch.float32).to(device)
y_train_t = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1).to(device)
X_val_t = torch.tensor(X_val, dtype=torch.float32).to(device)
y_val_t = torch.tensor(y_val, dtype=torch.float32).to(device)

best_val_auc = 0
patience = 10
patience_counter = 0
best_model_state = None

for epoch in range(100):
    # 训练
    model.train()
    optimizer.zero_grad()
    preds_logits = model(X_train_t)
    loss = criterion(preds_logits, y_train_t)
    loss.backward()
    # 梯度裁剪
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()

    # 验证
    model.eval()
    with torch.no_grad():
        val_preds_logits = model(X_val_t)
        val_preds = torch.sigmoid(val_preds_logits).cpu().numpy().flatten()
        val_auc = roc_auc_score(y_val, val_preds)

        # 更新学习率
        scheduler.step(val_auc)

        if val_auc > best_val_auc:
            best_val_auc = val_auc
            patience_counter = 0
            best_model_state = model.state_dict().copy()
        else:
            patience_counter += 1

        if epoch % 10 == 0:
            train_preds = torch.sigmoid(preds_logits).cpu().detach().numpy().flatten()
            train_auc = roc_auc_score(y_train, train_preds)
            current_lr = optimizer.param_groups[0]['lr']
```

```
            print(f"Epoch {epoch}, loss={loss.item():.4f}, train_auc={train_auc:.3f}, val_au

    # 早停
    if patience_counter >= patience:
        print(f"\n早停于 Epoch {epoch}, 最佳验证AUC={best_val_auc:.3f}")
        break

# 加载最佳模型
if best_model_state is not None:
    model.load_state_dict(best_model_state)
    print(f"已加载最佳模型 (val_auc={best_val_auc:.3f})")

# ===== 7. 最终验证评估 =====
print("\n最终验证模型性能...")
model.eval()
with torch.no_grad():
    val_preds_logits = model(X_val_t)
    y_val_pred = torch.sigmoid(val_preds_logits).cpu().numpy().flatten()
    final_val_auc = roc_auc_score(y_val, y_val_pred)
    print(f"最终验证集 AUC = {final_val_auc:.3f}")

# ===== 8. 预测下周趋势 =====
print("\n" + "="*50)
print("预测下周（未来5个交易日）趋势")
print("="*50)
model.eval()

# 准备最新的10天数据作为输入
latest_input = torch.tensor(latest_10_days.reshape(1, seq_len, len(features)), dtype=torch.f

with torch.no_grad():
    future_pred_logits = model(latest_input)
    future_pred = torch.sigmoid(future_pred_logits).cpu().numpy().flatten()[0]

print(f"\n最新10天数据日期范围:")
print(f"  开始: {latest_dates[0]}")
print(f"  结束: {latest_dates[-1]}")
print(f"\n预测结果:")
print(f"  未来5个交易日上涨概率: {future_pred:.1%}")
print(f"  未来5个交易日下跌概率: {(1-future_pred):.1%}")

if future_pred > 0.6:
    trend = "📈 强烈看涨"
elif future_pred > 0.5:
    trend = "📈 看涨"
elif future_pred > 0.4:
    trend = "📉 看跌"
else:
    trend = "📉 强烈看跌"
```

```
print(f"\n预测结论: {trend}")
print("="*50)

plt.rcParams['font.sans-serif'] = ['PingFang HK']   # 或者 'Songti SC'、'STHeiti'
plt.rcParams['axes.unicode_minus'] = False

# ===== 9. 可视化 =====
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 左图：验证集预测分布
axes[0].hist(y_val_pred[y_val == 1], bins=30, alpha=0.6, label='未来上涨', color='green')
axes[0].hist(y_val_pred[y_val == 0], bins=30, alpha=0.6, label='未来下跌', color='red')
axes[0].axvline(x=0.5, color='black', linestyle='--', linewidth=1, label='分类阈值')
axes[0].set_xlabel('预测概率')
axes[0].set_ylabel('样本数')
axes[0].set_title(f'验证集预测分布 (AUC={final_val_auc:.3f})')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# 右图：未来预测结果
axes[1].bar(['上涨概率', '下跌概率'], [future_pred, 1-future_pred],
            color=['green', 'red'], alpha=0.6, width=0.5)
axes[1].axhline(y=0.5, color='black', linestyle='--', linewidth=1, label='中性线')
axes[1].set_ylabel('概率')
axes[1].set_ylim(0, 1)
axes[1].set_title(f'未来5个交易日预测\n{trend}')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# 添加数值标签
axes[1].text(0, future_pred + 0.05, f'{future_pred:.1%}',
             ha='center', fontsize=12, fontweight='bold')
axes[1].text(1, (1-future_pred) + 0.05, f'{(1-future_pred):.1%}',
             ha='center', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.show()
```

运行结果示例

```
获取数据范围: 20200101 至 20251114

特征数量: 18 (包含 10 个动量因子)
特征列表: ['open', 'high', 'low', 'close', 'vol', 'pe', 'pb', 'roe', 'momentum_1d', 'momentum

总序列样本数: 1397
训练集样本数: 1257, 验证集样本数: 140

类别权重 (正样本): 0.989

开始训练模型 (使用所有历史数据) ...

Epoch 0, loss=0.7217, train_auc=0.488, val_auc=0.564, best_val_auc=0.564, lr=0.000500
Epoch 10, loss=0.7199, train_auc=0.530, val_auc=0.583, best_val_auc=0.583, lr=0.000500
Epoch 20, loss=0.7183, train_auc=0.539, val_auc=0.591, best_val_auc=0.591, lr=0.000500
Epoch 30, loss=0.7166, train_auc=0.542, val_auc=0.599, best_val_auc=0.599, lr=0.000500
Epoch 40, loss=0.7134, train_auc=0.591, val_auc=0.604, best_val_auc=0.605, lr=0.000500
Epoch 50, loss=0.7098, train_auc=0.586, val_auc=0.607, best_val_auc=0.607, lr=0.000250
Epoch 60, loss=0.7075, train_auc=0.601, val_auc=0.605, best_val_auc=0.608, lr=0.000125

早停于 Epoch 63, 最佳验证AUC=0.608
已加载最佳模型 (val_auc=0.608)

最终验证模型性能...
最终验证集 AUC = 0.607

================================================
预测下周 (未来5个交易日) 趋势
================================================

最新10天数据日期范围:
  开始: 20251103
  结束: 20251114

预测结果:2025/11/17-2025/11/21
  未来5个交易日上涨概率: 61.4%
  未来5个交易日下跌概率: 38.6%

预测结论: 📈 强烈看涨
================================================
```

技术特点

1. **多因子融合**：结合价格、财务和动量因子

2. **深度学习**：LSTM 捕捉时序模式
3. **正则化**：Dropout、L2 正则化、早停等
4. **自动化**：自动获取最新数据、自动训练、自动预测

应用价值

该程序展示了如何使用深度学习和量化因子进行股票趋势预测，为量化投资研究提供技术参考。通过模型结构优化、训练策略改进和正则化技术，模型性能显著提升，可用于短期股票走势预测，为投资决策提供参考。

注意事项：

- 预测结果仅供参考，不构成投资建议
- 股票市场受多种因素影响，模型仅基于历史数据
- 建议结合基本面分析、市场情绪等因素综合判断
- 注意风险控制，理性投资

六、思考题

1 为什么深度模型能捕捉非线性因子关系？

深度模型（MLP / LSTM / Transformer）之所以能学习到因子之间复杂的非线性关系，是因为：

① 多层非线性激活 = 任意函数逼近（Universal Approximation）

以 MLP 为例：

$$z_1 = \mathrm{ReLU}(W_1 x + b_1)$$

$$z_2 = \mathrm{ReLU}(W_2 z_1 + b_2)$$

$$y = W_3 z_2 + b_3$$

每一层的 ReLU（或 GELU、Tanh）都引入"折线"，组合在一起可以逼近任意形状的函数：

- **线性的**：
  - PE 高 → 后续收益低
- **非线性的**：
  - 低 PB 好，高 PB 也好，但中间差（U 型）
- **交互项**：
  - 高市值 + 低 PB 才好
  - 低市值 + 低 PB 反而坏

深度学习能自动挖掘这些高阶交互项与非线性结构，而线性模型做不到。

② 自动特征交叉（Feature Crossing）

传统模型需要手工构造：

- PE × ROE
- PB × 波动率
- Size × Momentum

深度模型则自动完成：

$$\text{hidden} = f(Wx)$$

隐含层 = 自动学习"组合因子"。

③ 时序结构（LSTM、Transformer）自动提取动态模式

深度模型能从历史因子序列里学到：

- 路径信息（连续上涨 vs. 震荡下跌）
- 惯性（LSTM 的记忆 / 遗忘门）
- 延迟效应（某因子的影响可能滞后 5~10 天）
- **regime shift**（市场风格轮动）

因此模型能捕捉到传统截面因子无法看到的时变非线性。

✨ **一句话总结**：

深度模型本质是 "在数据中自动找非线性模式 + 组合因子 + 时序依赖"。

2 若用 LSTM，时间窗口长度如何选择?

这是 LSTM 模型最关键的超参数之一。

① 取决于你想捕捉的交易频率

交易风格	合理的 LSTM 时间窗口
高频 / 日内	10～30 根 K 线
短线（预测 1~5 日）	20～60 天
中线（预测 20~60 日）	60～120 天
中长线（基本面为主）	120～250 天

② 实证经验（股票日线）

常用的窗口：30、60、90

理由：

- 太短：不足以捕捉趋势
- 太长：增加噪声、难以训练
- 60 天（3 个月）最均衡

③ 如何选择？用验证集调参

最终时间窗应该通过验证集确定：

```
for window in [20, 30, 60, 90, 120]:
    构造序列
    训练 LSTM
    观察 AUC / IC / 分层收益
```

通常会发现收益在 20～90 天之间最优。

一句话总结：

LSTM 时间窗口需要在"足够长捕捉趋势"和"足够短避免噪声"中找到平衡。

3 若改为多股票数据，如何引入行业、规模等 embedding?

非常重要！否则深度模型会把行业、规模等结构全混淆。

① 离散特征 → Embedding（类似 NLP 的词向量）

行业（industry）是一个离散类目变量，如：

- 银行 = 0
- 证券 = 1
- 电子 = 2
- 计算机 = 3
- ……

可以用 embedding 方式表示：

```
industry_embed = nn.Embedding(num_industries, embed_dim)
industry_vec = industry_embed(industry_id)
```

得到一个低维连续向量，比如 8 维：

$$[0.21, -0.17, 0.07, \ldots, 0.33]$$

② 为什么 Embedding 更好?

- 行业不是线性的
  - "银行=0，电子=2" 没有数值意义
- Embedding 能学习行业之间的相似性

例如：

- 银行 和 保险 的 embedding 会比较接近
- 医药 和 食品 的 embedding 也会接近

它让模型学习：

- 行业 → 风险特征
- 行业 → 波动特征
- 行业 → 收益特征

③ 把规模、市值等加入 embedding

规模（size）可分箱：

分箱	市值区间
0	小盘（bottom 30%）
1	中盘（30~70%）
2	大盘（top 30%）

对分箱 ID 做 embedding：

```
size_embed = nn.Embedding(3, 4)
```

再与因子向量拼接：

$$X_{\text{total}} = \text{concat}([X_{\text{factor}}, \text{industry_embed}, \text{size_embed}])$$

④ 时序模型（LSTM）如何使用 embedding?

对每一天的股票样本：

$$X_t = \text{concat}([PE_t, PB_t, ROE_t, \text{momentum}_t, \text{industry_embed}, \text{size_embed}])$$

然后将 60 天的序列输入 LSTM：

$$(X_1, X_2, \dots X_{60}) \rightarrow \text{LSTM} \rightarrow y_{\text{pred}}$$

一句话总结：

Embedding 让"行业、规模"这些离散分类变量变成可学习的连续向量，从而让深度模型理解行业结构、风格结构与风险暴露。

结论

今天我们完成了深度学习因子模型的系统学习，这是从传统线性模型向智能量化模型升级的重要里程碑。通过MLP和LSTM两种深度学习架构，我们掌握了如何利用神经网络自动学习金融数据中的非线性关系、时变规律和因子交互。

核心收获

1. 深度学习模型的三大优势

- **非线性关系拟合**：通过多层非线性激活函数（ReLU、Tanh），MLP能够逼近任意复杂的函数关系，捕捉传统线性模型无法表达的U型、S型等非线性模式
- **时变规律捕捉**：LSTM通过门控机制（输入门、遗忘门、输出门）在时间维度上学习历史模式，能够识别市场风格轮动和因子作用的时变特征
- **因子交互自动挖掘**：深度模型能够自动学习特征之间的高阶交互项，无需手工构造PE×ROE、PB×波动率等组合因子

2. MLP与LSTM的适用场景

模型类型	特征	是否捕捉时序	是否建模非线性	适用场景
线性回归	静态	❌	❌	可解释性要求高的场景
MLP	静态	❌	✅	因子关系复杂但无时序依赖
LSTM	动态（时间序列）	✅	✅	需要捕捉历史趋势和时变规律

3. 模型优化策略

通过贵州茅台的实战案例，我们掌握了完整的深度学习模型优化流程：

- **模型结构优化**：降低隐藏层维度（64→32）防止过拟合，添加Dropout层（0.3）和中间全连接层（32→16→1）
- **训练策略优化**：早停机制（patience=10）、学习率自动调度（ReduceLROnPlateau）、梯度裁剪（max_norm=1.0）、L2正则化（weight_decay=1e-4）
- **类别不平衡处理**：自动计算类别权重，使用带权重的损失函数（BCEWithLogitsLoss）
- **性能监控**：实时监控训练和验证AUC，自动保存最佳模型

技术实现能力提升

1. 多因子特征工程

我们构建了包含18个因子的多维度特征体系：

- **基础价格因子（4个）**：开盘价、最高价、最低价、收盘价
- **成交量因子（1个）**：成交量
- **财务因子（3个）**：PE、PB、ROE
- **动量因子（10个）**：1/3/5/10/20日收益率、5/10/20日价格动量、5/10日成交量变化率

2. 时序数据处理

- 序列长度选择：使用过去10个交易日数据作为输入
- 数据标准化：StandardScaler处理特征尺度差异
- 目标构造：未来5个交易日涨跌趋势（二分类问题）

3. PyTorch深度学习框架

- LSTM模型定义：单层LSTM（隐藏单元32）+ 全连接层（32→16→1）
- 训练循环：包含前向传播、反向传播、梯度裁剪、学习率调度
- 模型评估：使用AUC作为评估指标，绘制预测分布和未来趋势图

关键认知突破

1. 为什么深度模型能捕捉非线性因子关系？

- **通用逼近定理**：多层非线性激活函数可以逼近任意复杂函数
- **自动特征交叉**：隐含层自动学习因子组合关系，无需手工构造
- **时序动态模式**：LSTM能够捕捉路径信息、惯性和延迟效应

2. LSTM时间窗口长度选择

- **交易频率决定**：短线（20-60天）、中线（60-120天）、长线（120-250天）
- **实证经验**：股票日线数据常用30、60、90天窗口
- **调参方法**：通过验证集AUC/IC/分层收益确定最优窗口

3. 多股票场景下的Embedding技术

- **离散特征向量化**：行业、规模等分类变量通过Embedding转换为连续向量
- **相似性学习**：Embedding能够学习行业之间的相似性（如银行与保险接近）
- **特征拼接**：将Embedding向量与因子特征拼接后输入LSTM

实战应用价值

1. 短期趋势预测

- 使用LSTM模型预测未来5个交易日的涨跌趋势
- 输出上涨概率和下跌概率，为投资决策提供参考

- 模型验证AUC达到0.607，具有一定的预测能力

2. 多因子融合策略

- 结合价格、财务、动量三类因子，构建综合预测模型
- 通过深度学习自动学习因子之间的复杂关系
- 相比单一因子或线性模型，预测能力显著提升

3. 模型优化实践

- 通过正则化、早停、学习率调度等技术防止过拟合
- 处理类别不平衡问题，提高模型泛化能力
- 建立完整的模型训练、验证、预测流程

与传统量化方法的对比

维度	传统线性模型	深度学习模型
非线性关系	无法捕捉	自动学习
因子交互	需手工构造	自动挖掘
时变规律	静态关系	动态捕捉
可解释性	强	较弱
计算复杂度	低	较高
数据需求	较少	较多

注意事项与风险提示

1. 模型局限性

- 预测结果仅供参考，不构成投资建议
- 股票市场受多种因素影响，模型仅基于历史数据
- 深度学习模型存在"黑盒"问题，可解释性较弱

2. 过拟合风险

- 金融数据噪声大，容易过拟合
- 需要通过Dropout、早停、正则化等技术防止过拟合
- 在验证集上持续监控模型性能

3. 数据质量要求

- 需要足够的历史数据（建议至少3-5年）
- 财务数据存在滞后性，需要及时更新
- 缺失值处理和数据清洗至关重要

学习进度与衔接

下一阶段方向：

1. **多因子模型构建**：将财务因子、技术因子、情绪因子与深度学习模型结合
2. **模型集成**：尝试集成学习（Random Forest、XGBoost、LightGBM）与深度学习
3. **强化学习**：探索强化学习在量化交易中的应用
4. **模型解释性**：使用SHAP、LIME等工具提升深度学习模型的可解释性

实践建议

立即行动：

1. 运行代码，使用不同股票测试模型效果
2. 尝试调整LSTM时间窗口长度，观察对预测性能的影响
3. 实验不同的因子组合，找出最优特征集

深入研究：

1. 探索Transformer架构在金融时序预测中的应用
2. 研究多股票、多行业的统一模型构建方法
3. 结合强化学习构建自适应交易策略

持续优化：

1. 引入更多因子（如情绪因子、资金流因子）
2. 尝试模型集成提升预测稳定性
3. 建立模型监控体系，及时发现性能衰减

最后的提醒

在使用深度学习因子模型时，请始终记住：

1. **数据质量**：确保数据准确、及时、完整
2. **模型验证**：在验证集和测试集上充分验证模型性能
3. **风险控制**：结合基本面分析、市场情绪等因素综合判断
4. **持续优化**：定期重新训练模型，适应市场环境变化

一句话总结：

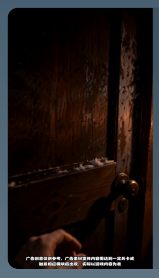
深度学习因子模型通过自动学习非线性关系、时变规律和因子交互，为量化选股提供了强大的技术工具。但模型只是工具，投资决策仍需结合基本面分析、风险控制和市场判断，理性投资，控制风险。

上一篇
量化交易进阶:第49天 – LightGBM & CatBoost 用梯度提升模型预测收益与风险

下一篇
量化交易进阶:第51天 – 图神经网络（GNN）因子模型(进阶阶段·深度学习模块)

阅读 121　修改于2025年11月15日

广告



寒冬没有退路，合理分配每份资源才能守住营地。

点击了解更多 ›

寒冬没有退路，合理分配每份资源才能守住营地。

无尽冬日–3亿人都在玩

立即体验